

Magnitude, Distance and Scalar Operations on Vectors

Contents

1. Overview	2
2. Magnitude: the distance from the origin	3
2.1 In 2D and 3D	3
2.2 In n dimensions	4
3. Euclidean distance	5
3.1 Distance as the norm of the difference	5
3.2 Distance in ML: the nearest neighbour	6
4. Operations with a scalar	7
4.1 Adding or subtracting a scalar: shifting	8
5. Mean centring: shifting in ML	9
6. Multiplying or dividing by a scalar: scaling	10
7. Summary	13
8. Sources	13
9. Key terms	13

Where this fits: Pipeline map step 0 (Foundations). See the [Course map](#).



- **Builds on:** Vectors and feature vectors ([Note MA-048](#)).
- **Leads to:** Regularisation ([Note DL-026](#)).
- **Compare with:** Cosine similarity ([Note MA-050](#)).

1. Overview

Key point: Pythagoras' theorem gives both the length of a vector and the distance between two vectors, in any number of dimensions; adding or multiplying by a scalar shifts or scales a vector.

Magnitude: distance from the origin Euclidean distance between two points

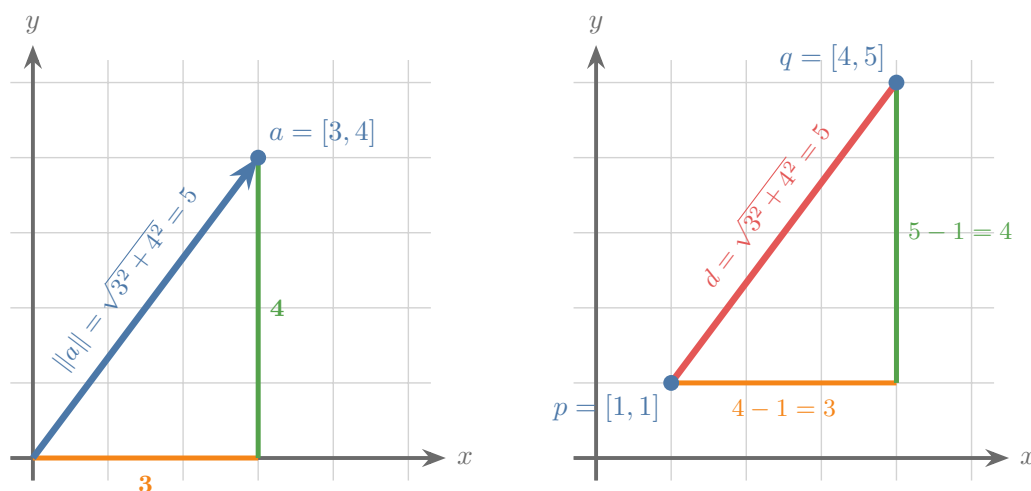


Figure 1: Left: the length of a vector. Right: the distance between two points. Both are Pythagoras' theorem

Figure 1 shows the two measurements this Note builds on. The length of $[3, 4]$ and the distance from $[1, 1]$ to $[4, 5]$ are both 5, and both come from the same right triangle. This Note:

- works out the length in 2D, 3D and n dimensions (section 2);
- does the same for the distance, and shows where ML uses it (section 3);
- turns to the simplest operation on a vector, adding a scalar (section 4), and its main use in ML, mean centring (section 5);
- ends with multiplying by a scalar (section 6).

A **vector** is a list of numbers such as $[3, 4]$; each number is a **component**, and the number of components is the vector's **dimension**. A **scalar** is a single number such as 5. All are defined in [what a vector is](#).

2. Magnitude: the distance from the origin

Key point: The magnitude of a vector is the square root of the sum of its squared components; the same formula works in every dimension.

The **magnitude** of a vector is its distance from the origin: the length of the arrow from its tail to its head. The magnitude is also called the length or the norm of the vector (the name used in [the margin in terms of \$w\$](#)), and written $\|x\|$.

2.1 In 2D and 3D

Key point: In 2D the two components are the legs of a right triangle, so Pythagoras gives the length.

In Figure 1 (left), the vector $a = [3, 4]$ forms a right triangle with the x-axis: one leg is 3 long, the other 4. The arrow is the hypotenuse, so by Pythagoras' theorem its length is:

$$\sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

For a general 2D vector $[a, b]$ the length is:

$$\sqrt{a^2 + b^2}$$

The same logic holds in 3D, with Pythagoras used twice (Figure 2). For $[2, 3, 6]$, the floor diagonal under the arrow joins $[0, 0, 0]$ to $[2, 3, 0]$ and has this length:

$$\sqrt{2^2 + 3^2} = \sqrt{13}$$

That diagonal and the height 6 are the legs of a second right triangle, whose hypotenuse is the arrow:

$$\sqrt{13 + 36} = \sqrt{49} = 7$$

In general a vector $[a, b, c]$ has this length:

$$\sqrt{a^2 + b^2 + c^2}$$

It is the square root of the sum of the squares of all its components.

$$\|[2, 3, 6]\| = \sqrt{2^2 + 3^2 + 6^2} = \sqrt{49} = 7$$

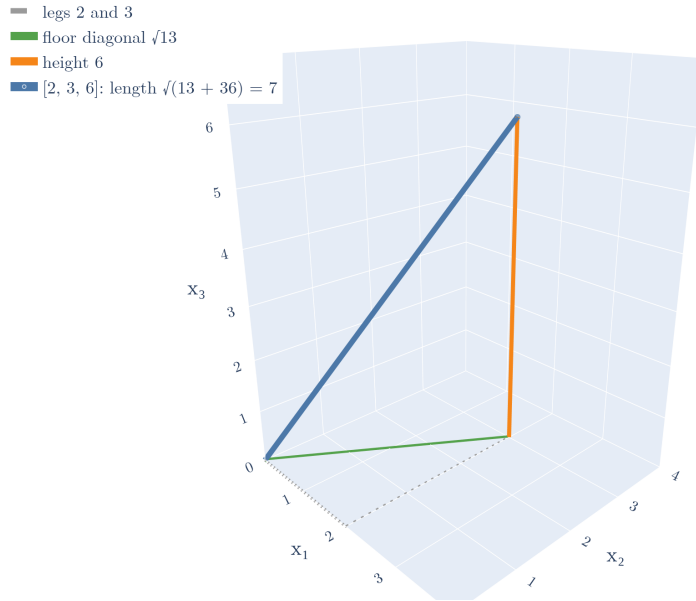


Figure 2: The length of $[2, 3, 6]$ by Pythagoras twice: the floor diagonal (green) has length the square root of 13; with the height 6 (orange) it forms a right triangle whose hypotenuse, the vector (blue), has length 7.

2.2 In n dimensions

Key point: Square every component, add, take the square root.

A ruler measures an arrow drawn in 2D or 3D. A vector with 50 components cannot be drawn, so its length has to come from a rule, and the rule must agree with Pythagoras wherever we can draw.

1. **In words:** square every component, add the squares, and take the square root.
2. **Formula:** for $x = [x_1, x_2, \dots, x_n]$,

$$\|x\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

$$\|x\| = \sqrt{\sum_{i=1}^n x_i^2}$$

Here $\sum_{i=1}^n$ means: add the terms for $i = 1, 2, \dots, n$.

3. **Example:** for the 3D vector $[2, 3, 6]$,

$$\|x\| = \sqrt{4 + 9 + 36} = \sqrt{49} = 7$$

and for the 5D vector $[1, 2, 3, 4, 5]$,

$$\|x\| = \sqrt{1 + 4 + 9 + 16 + 25}$$

$$\|x\| = \sqrt{55} \approx 7.42$$

The sum under the square root is the vector multiplied with itself, component by component. That sum is the dot product of the vector with itself, taught in [computing the dot product](#), so:

$$\|x\|^2 = x \cdot x$$

For $[2, 3, 6]$:

$$2 \times 2 + 3 \times 3 + 6 \times 6$$

$$4 + 9 + 36 = 49 = 7^2$$

Python: NumPy computes the magnitude of a vector of any dimension with one function.
`import numpy as np`

```
v = np.array([1, 2, 3, 4, 5])
np.linalg.norm(v)      # 7.416...
```

`np.linalg` is NumPy's linear algebra module. The same call works unchanged for a 15-dimensional or a 500-dimensional vector.

Extra: The magnitude is one norm among several, the **L2 norm** (G-1028); adding the absolute values instead, $|x_1| + \dots + |x_n|$, gives the **L1 norm** (G-1025). Ridge regression penalises the squared L2 norm of the coefficients and Lasso their L1 norm (see [Ridge penalising large coefficients](#) and [Lasso on one feature](#)), so vector norms are the building block of regularisation.

3. Euclidean distance

Key point: The Euclidean distance between two vectors is the magnitude of their difference: subtract component by component, then take the norm.

The **Euclidean distance** (G-715), the straight-line distance between two points, is defined in [the Euclidean distance](#). Here we see how it links to the magnitude.

3.1 Distance as the norm of the difference

Key point: Subtract the two vectors, then take the magnitude of the result.

In Figure 1 (right) the legs of the triangle, 3 and 4, are the components of the difference vector $q - p$:

$$q - p = [4, 5] - [1, 1]$$

$$q - p = [3, 4]$$

The distance is the hypotenuse, which is exactly the magnitude of that vector. Subtracting the other way round, $p - q = [-3, -4]$, only flips the signs, and squaring removes them, so the order does not matter:

$$d(p, q) = \|p - q\|$$

The squared differences in the distance formula are the squared components of $p - q$. So for $p = [6, 7, 8, 9, 10]$ and $q = [1, 2, 3, 4, 5]$ the difference is $[5, 5, 5, 5, 5]$ and the distance is the square root of five squares of 5, one step per line:

$$5^2 + 5^2 + 5^2 + 5^2 + 5^2 = 125$$

$$\sqrt{125} \approx 11.18$$

Python: The distance is two steps: subtract, then `norm`.

```
p = np.array([6, 7, 8, 9, 10])
q = np.array([1, 2, 3, 4, 5])
np.linalg.norm(p - q)    # 11.18...
```

This works for vectors of any dimension, as long as both have the same number of components.

3.2 Distance in ML: the nearest neighbour

Key point: Many algorithms decide by distance: a new point gets the class of the vectors nearest to it.

A lot of ML algorithms compute Euclidean distances inside. The clearest example is [K-nearest neighbours](#) (KNN, G-998), a classification algorithm. To classify a new iris flower from its sepal length and petal length, KNN treats the flower as a vector, computes its distance to every flower in the data, and gives it the species of the nearest ones.

The query takes the class of its nearest vector

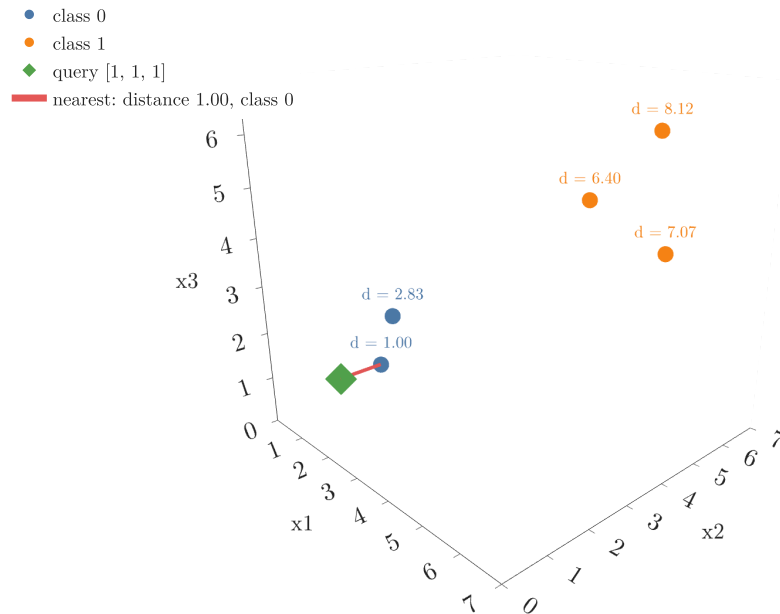


Figure 3: Five labelled vectors in 3D and a query vector; the query takes the class of the nearest one

Figure 3 runs this on five 3D vectors in two classes. The query vector $[1, 1, 1]$ is 1.00 from $[1, 2, 1]$ and at least 2.83 from all others. Its nearest neighbour is in class 0, so the query is classed 0.

The same distance appears in [K-means clustering](#), which groups points around their nearest centre, and in recommender systems, as in [recommending by distance](#).

4. Operations with a scalar

Key point: A scalar operation applies the same number to every component; adding or subtracting a scalar shifts the vector.

A vector and a scalar can be combined in two ways: by adding (or subtracting, which works the same way) and by multiplying (or dividing). In both, the scalar is applied to every component in turn.

4.1 Adding or subtracting a scalar: shifting

Key point: Adding a scalar adds it to every component, which moves the point; this is called shifting.

1. **In words:** add the scalar to every component of the vector.

2. **Formula:** for $v = [v_1, v_2, \dots, v_n]$ and a scalar s ,

$$v + s = [v_1 + s, v_2 + s, \dots, v_n + s]$$

3. **Example:**

$$[2, 2] + 5 = [7, 7]$$

$$[2, 3] + 3 = [5, 6]$$

Subtraction works the same way, with s subtracted from every component:

$$[2, 3] - 1 = [1, 2]$$

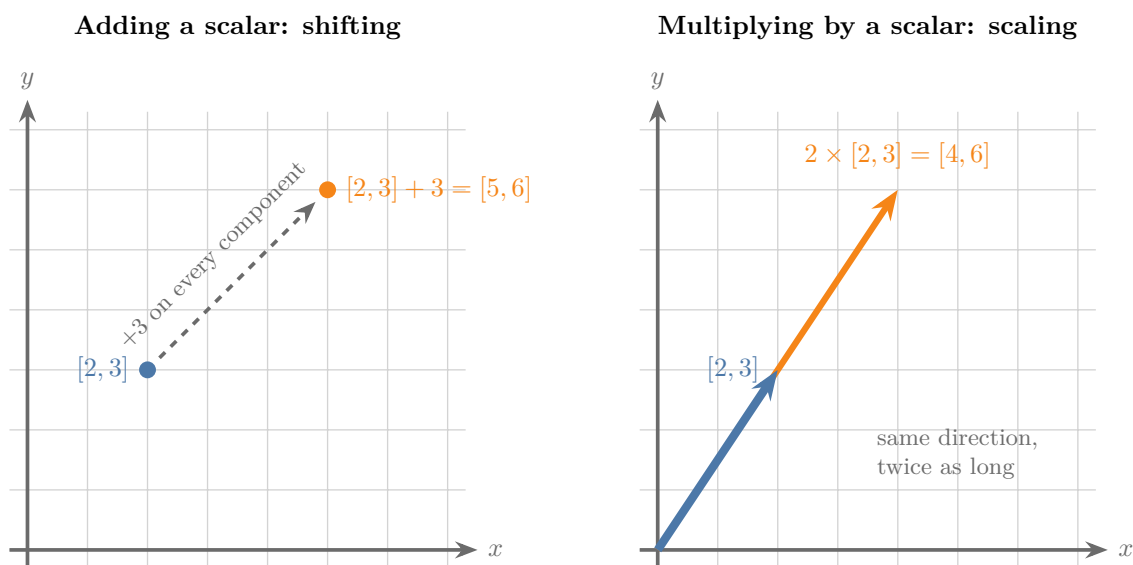


Figure 4: Left: adding a scalar moves the point. Right: multiplying by a scalar stretches the arrow

Changing every component moves the point to a new place in the coordinate system (Figure 4, left). So adding or subtracting a scalar is called **shifting** (G-1791).

Extra: Strictly, adding a scalar to a vector is not an operation of linear algebra; a vector space has only two operations, adding two vectors and multiplying a vector by a scalar (MML Definition 2.9). NumPy allows it through **broadcasting** (G-333): in the words of the NumPy user guide (NumPy, “Broadcasting”), the scalar is “stretched” into an array

of the same shape, here the vector $[s, s, \dots, s]$, and that is added. So $[2, 3] + 3$ is really $[2, 3] + [3, 3]$.

5. Mean centring: shifting in ML

Key point: Mean centring is a scalar subtraction on vectors: each feature minus its own mean.

Mean centring (G-1195), the first half of standardization (see [what standardization does to the data](#)), slides the cloud of points until its mean sits at the origin, without changing its shape. As a vector operation, mean centring is shifting. Each **feature** (G-772; an input variable, one column of the data table) is a vector of values, and its mean is a scalar. Mean centring subtracts that scalar from every component, the way moving every house on a street by the same distance leaves the street's shape unchanged. For the feature $[3, 5, 7]$, with mean 5:

$$[3, 5, 7] - 5 = [-2, 0, 2]$$

Figure 5 does this to the 150 iris flowers, with their sepal length and sepal width as two features. Every flower moves by the same vector, minus the mean $(5.84, 3.06)$, so the cloud slides as one piece until its mean sits on the origin; its shape and spread do not change.

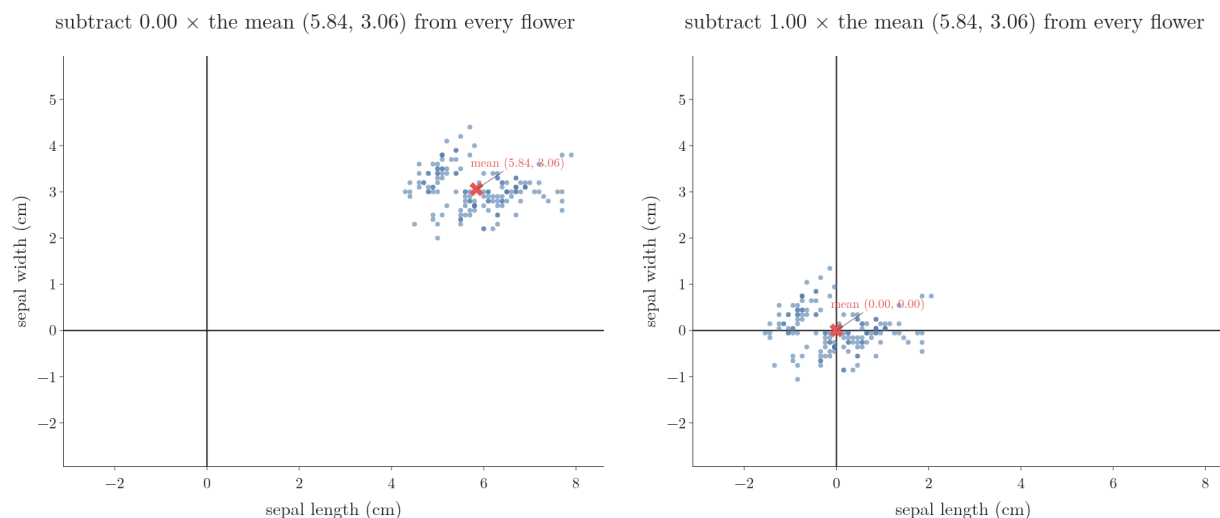


Figure 5: The 150 iris flowers (sepal length and width) shifted by a growing fraction of their mean, until the mean (red cross) sits at the origin. The cloud keeps its shape.

Python: Mean centring is one line: subtracting a row of means from a table subtracts each mean from its own column.

```
data = np.array([[3, 4], [5, 6], [7, 8]])
centred = data - data.mean(axis=0) #  $[[ -2, -2], [0, 0], [2, 2]]$ 
axis=0 takes the mean down each column. The Notebook for this Note
```

(MA-049-magnitude-distance-and-scalar-operations.ipynb) runs every example of this Note, plus a 100-vector mean-centring plot and the nearest-neighbour demo with a query you can change.

6. Multiplying or dividing by a scalar: scaling

Key point: Multiplying by a scalar multiplies every component, which stretches or shrinks the arrow without turning it; a negative scalar also flips it.

The second scalar operation works like the first, with multiplication in place of addition. Figure 4 (right) and Figure 6 show it on the vector $[2, 3]$, whose length is $\sqrt{13} \approx 3.61$. We take the scalars one at a time.

1. A scalar above 1 stretches.

$$2 \times [2, 3] = [4, 6]$$

The new arrow lies on the same line through the origin and points the same way, but is twice as long: 7.21.

2. A scalar between 0 and 1 shrinks.

$$0.5 \times [2, 3] = [1, 1.5]$$

Same direction, half the length. Dividing by s is the same as multiplying by $1/s$, so this is also $[2, 3]/2$.

3. The scalar -1 flips.

$$-1 \times [2, 3] = [-2, -3]$$

The length stays 3.61, but the arrow now points the opposite way.

4. A negative scalar flips and scales.

$$-2 \times [2, 3] = [-4, -6]$$

The opposite direction, and twice as long.

The number line under Figure 6 shows why a negative scalar flips. Multiplying the number 1 by 2 moves it out to 2, on the same side of zero. Multiplying it by -1 sends it to -1 , the same distance from zero on the other side. A vector behaves the same way along its own line through the origin.

The operation is called **scaling** (G-1746), and the name **scalar** (G-1743) comes from it: a scalar is a number that scales a vector.

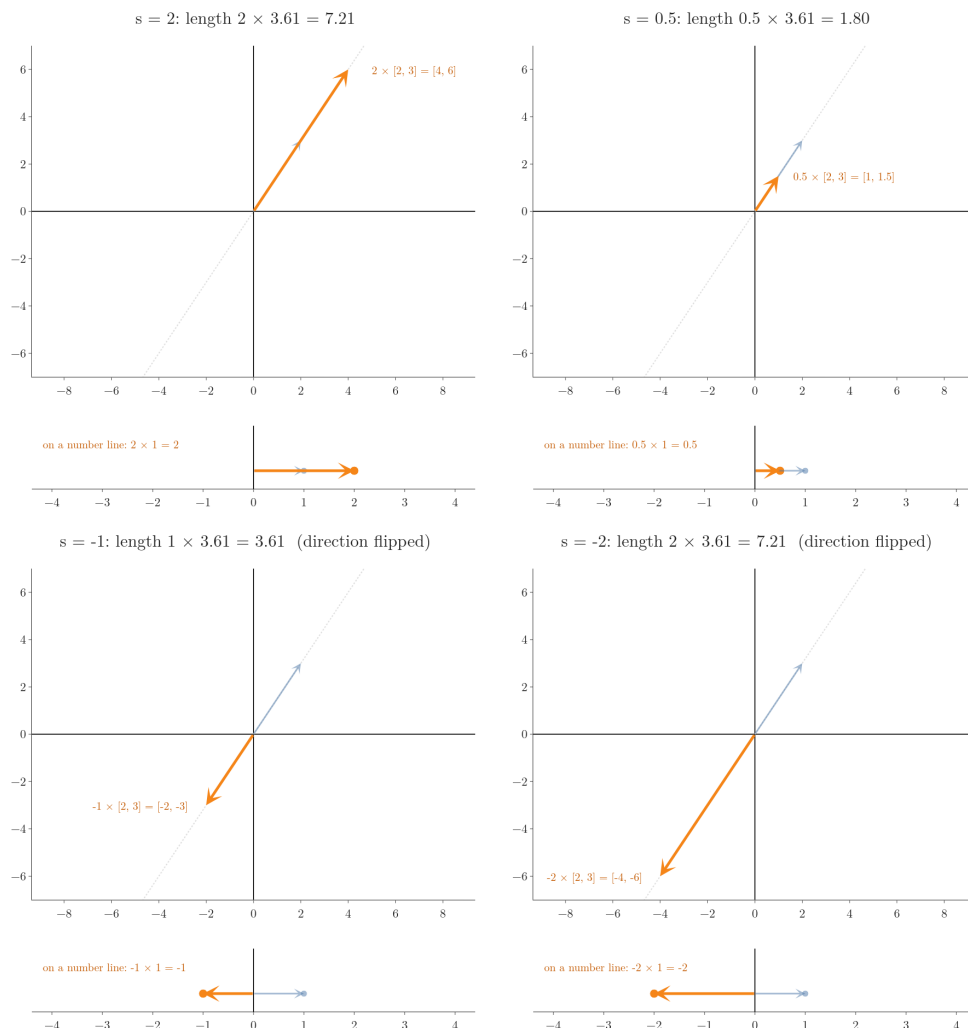


Figure 6: The vector $[2, 3]$ (pale blue) multiplied by 1, 2, 0.5, -1 and -2 (orange). Every result lies on the same line through the origin; the length is multiplied by the size of the scalar, and a negative scalar flips the direction. Below, the same scalar acts on the number 1 on a number line. Number-line idea after Khan Academy, “Multiplying a vector by a scalar”.

The formal version.

1. **In words:** multiply every component by the scalar.
2. **Formula:**

$$s v = [s v_1, s v_2, \dots, s v_n]$$

3. **Effect on the length:** the magnitude is multiplied by $|s|$, the **absolute value** of s (its size with the sign dropped, so $|-2| = 2$ and $|0.5| = 0.5$). For $x = [2, 3]$ and $s = -2$, we get $sx = [-4, -6]$, with length:

$$\sqrt{16 + 36} = \sqrt{52} = 2\sqrt{13}$$

This is twice the length $\sqrt{13}$ of x . In general:

$$\|sx\| = \sqrt{s^2x_1^2 + \cdots + s^2x_n^2}$$

$$\|sx\| = |s| \|x\|$$

Dividing a vector by its own magnitude scales it to length 1, giving the **unit vector** (G-2048) used in [projecting one point](#). For $[3, 4]$, whose length is 5:

$$[3, 4]/5 = [0.6, 0.8]$$

7. Summary

Idea	Formula	Example
Magnitude	$\ x\ = \sqrt{\sum x_i^2}$	$\ [3, 4]\ = 5$
Euclidean distance	$\ p - q\ $	$[1, 1]$ to $[4, 5]$: 5
Shifting	$v + s$: add s to every component	$[2, 3] + 3 = [5, 6]$
Mean centring	each feature minus its mean	$3, 5, 7 \rightarrow -2, 0, 2$
Scaling	sv : multiply every component by s	$2 \times [2, 3] = [4, 6];$ $-1 \times [2, 3] = [-2, -3]$

- Pythagoras' theorem gives both the length of a vector and the distance between two vectors, in any dimension.
- Distance = magnitude of the difference: `np.linalg.norm(p - q)`.
- KNN, K-means and recommender systems all decide by distance.
- Shifting moves a vector, scaling stretches it; mean centring is shifting applied to whole features.

8. Sources

Built from

- CampusX, “Supercharge Your ML Journey: Mastering Vectors in Linear Algebra - Part 1”, YouTube, <https://www.youtube.com/watch?v=mQewAJb8oJ8>
- Khan Academy, “Multiplying a vector by a scalar | Vectors and spaces | Linear Algebra | Khan Academy”, YouTube, <https://www.youtube.com/watch?v=ZN7YaSbY3-w>
- Khan Academy, “Vector dot product and vector length | Vectors and spaces | Linear Algebra | Khan Academy”, YouTube, https://www.youtube.com/watch?v=WNuIhXo39_k

Other references

- Deisenroth, M. P., Faisal, A. A. and Ong, C. S. (2020). *Mathematics for Machine Learning* (MML). Cambridge University Press. Definition 2.9 (Vector Space).
- NumPy developers. *NumPy User Guide*, “Broadcasting”. [numpy.org, basics.broadcasting.html](https://numpy.org/basics/broadcasting.html).

9. Key terms

Term	Meaning
Magnitude (norm, length) of a vector (G-1028)	Its distance from the origin: $\sqrt{x_1^2 + \dots + x_n^2}$
L2 norm (G-1028)	The usual magnitude: square root of the sum of squared components

Term	Meaning
L1 norm	The sum of the absolute values of the components
Shifting	Adding or subtracting a scalar to every component of a vector
Broadcasting	NumPy stretching a scalar (or smaller array) to match a bigger array before an operation
Scaling (a vector)	Multiplying or dividing every component of a vector by a scalar